№8 ООП

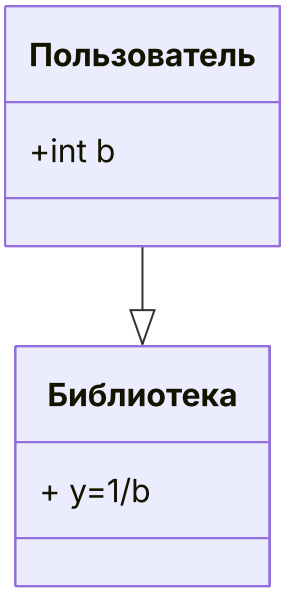
План

- Обработка исключительных ситуаций
  - Стратегии обработки ошибок

Обработка исключительных ситуаций

Ошибки делятся на две категории:

- *Статические* - их можно обнаружить на этапе написания кода.
- *Динамические* - ошибки возникающие непосредственно в время выполнения программы.



Тут может возникнуть ошибка деления на ноль и она динамическая. Что бы исправить подобное было введено понятие *особой ситуации*.

Допустим у нас есть класс:

```
class Vector {
    int *p;
    int sz;
```

```
    int operator[](int i);
    class Range {} // Класс исключения
}
```

Описываем исключение:

```
// Реализация возвращения ошибки
int& Vector::operator[](int)
{
    if(0≤i&&i<sz) return p[i];
    throw Range(); // прямой вызов исключения
}

// Обработка исключения
try{ // Перехват возможного сключения
    Vector v;
    v[-1]
} catch(Vector::Range) { // catch(<тип исключения>) -
обработка исключения заданого класса
    throw; // перевызов исключения. Он нужен для передачи
ошибки на уровень выше для того, что бы центролизовать
обработку ошибок.
}
```

Стратегии обработки ошибок

Их всего три:

1. *Завершить программу* - плохой вариант, просто `terminate()`.
2. *Вернуть код ошибки* - уже лучше, вернуть код ошибки для последующего дебага.
3. *Вернуть допустимое значение* - самый плохой вариант. Поведение программы становится непредсказуемым.
4. *Исключения* - лучший вариант, можно будет обработать ошибку управляемо.

Обработка исключений хорошо *считается с наследованием и полиморфизмом*, поскольку *можно* наследовать класс ошибок и

таким образом *получить семейство исключений* для нескольких классов. При наследовании обязательно нужно использовать константные ссылки в `catch(const v&)` иначе он может обратиться к родительскому классу и срезать поля наследника.